

Hochschule Ravensburg-Weingarten
University of Applied Sciences



Project Report

Real-Time Digital Filter Evaluation Platform

Author: Yash Balu Sangale (29346383)

Co-Author: Neel Karia (16346352)

Study Program: M.Eng. Electrical Engineering and Embedded
Systems

Faculty of Electrical Engineering and Computer Science

Supervisor: Prof. (Dr.) Christoph Ellenrieder

Revision: 1.0

Date: August 14, 2026

Contents

1	Introduction	1
1.1	Background	1
1.2	Problem Statement	1
1.3	Objectives	1
1.4	Implemented Solution	2
2	System Design and Implementation	3
2.1	System Architecture	3
2.2	ESP32-S3 Platform Controller	4
2.3	Execution Modes	4
2.4	Test Signal Generation	5
2.5	Real-Time Sampling and Processing	5
2.6	Digital Filter Implementation	5
2.6.1	Internal Processing (ESP32-S3)	6
2.6.2	External Target Processing (Arduino Nano 33 IoT)	6
2.7	Target Operating States	7
2.8	Runtime Filter Configuration	7
2.9	Inter-Controller Communication	8
2.10	DSM Telemetry Packet	8
2.11	UDP Telemetry Transmission	9
2.12	Host-Side Visualization	9
3	Results	10
3.1	Filter Design Verification	10
3.2	Real-Time Visualization Interface	10
3.3	Real-Time Filter Evaluation	11
3.4	Limit-Cycle Behaviour	13
3.5	Execution Time and CPU Load	13
3.6	Microcontroller Comparison	14
3.7	Comparison of Real-Time Realizations	15
3.8	Discussion	15
3.9	Conclusion	16

List of Figures

1.1	Hardware setup of the developed real-time digital filter evaluation platform.	2
2.1	Overall architecture of the real-time digital filter evaluation platform. . .	3
3.1	Real-time graphical interface for signal visualization and statistical analysis.	11
3.2	Real-time impulse response of the ninth-order floating-point Direct Form I IIR Bessel filter.	12
3.3	Zoomed view of the discrete-time impulse response of the floating-point Direct Form I IIR implementation.	12
3.4	Measured signal statistics during the floating-point IIR impulse-response test.	13

List of Tables

2.1	Runtime command interface of the ESP32-S3 platform.	7
2.2	Runtime command interface. <code>imode</code> , <code>freq</code> , <code>amp</code> , <code>wave</code> , and <code>gcoeff</code> apply to the ESP32-S3 only; the target implements <code>fmode</code> , <code>scoeff</code> , <code>reset</code> , and <code>stop/start</code>	7
2.3	DSM packet layout. The total packet size is 20 B in UDP mode (with <code>seq</code>) and 18 B in USB mode (without <code>seq</code>), as verified by a compile-time size assertion.	8
3.1	Execution time and CPU load of the supported filter realizations on the ESP32-S3 for $N = 9$ and $T_s = 200 \mu\text{s}$	14
3.2	Reference execution performance for selected ninth-order filter realizations on different microcontroller platforms.	15

Chapter 1

Introduction

1.1 Background

Digital filters are a core building block of embedded signal processing, used to remove noise, shape frequency responses, and condition signals prior to further analysis or control. Filter design is conventionally a two-stage process: the filter is first evaluated offline in simulation (e.g. MATLAB), and the resulting implementation is then transferred to target hardware for verification. This separation makes it difficult to directly compare simulated and measured behaviour during development, and it does not expose implementation-specific effects such as fixed-point quantization, coefficient scaling error, execution-time variation, and sampling jitter. A real-time evaluation platform that observes the filter during on-target execution addresses this gap.

1.2 Problem Statement

Evaluating a digital filter on embedded hardware is constrained by three practical limitations:

- **Lack of real-time observability:** filter output running on a microcontroller is not visible to the developer unless it is explicitly captured and transmitted to an external system.
- **Reconfiguration overhead:** changing the filter type or coefficients on a conventional setup requires firmware modification and re-flashing, interrupting the test-and-observe cycle.
- **No unified visualization layer:** even when signal data is captured, a lightweight mechanism for synchronized multi-channel visualization with runtime-adjustable display parameters (sample window, channel visibility) is typically unavailable.

1.3 Objectives

The project implements a Real-Time Digital Filter Evaluation Platform that:

1. generates a known test signal and applies it to a digital filter under test, executed either locally or on a dedicated target microcontroller;

2. acquires the resulting signals and streams them to a host computer without manual data retrieval;
3. provides synchronized, browser-based visualization of the acquired channels with runtime-configurable display parameters.

1.4 Implemented Solution

The platform is built around an ESP32-S3, which performs signal generation and acquisition and acts as the network bridge. A separate Arduino Nano 33 IoT target microcontroller optionally executes the digital filter under test. Processed and acquired data are transmitted wirelessly via UDP to a Node.js host bridge and forwarded to a browser-based interface over WebSocket. This establishes a continuous development loop in which the signal is generated, processed on real hardware, transmitted, and visualized without manual intervention.

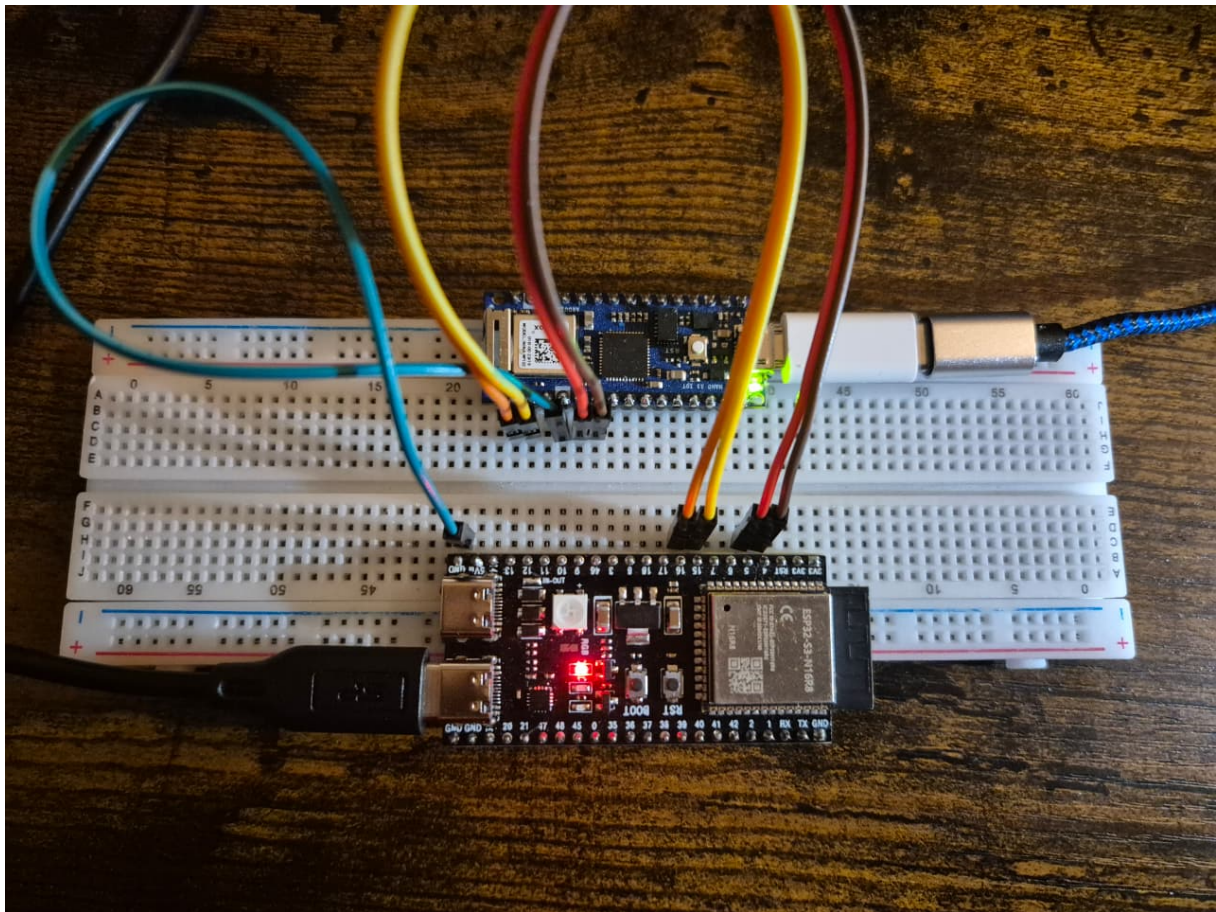


Figure 1.1: Hardware setup of the developed real-time digital filter evaluation platform.

The system architecture, implementation, and evaluation are described in Chapter 2 and Chapter 3.

Chapter 2

System Design and Implementation

The evaluation platform combines an ESP32-S3 platform controller, an Arduino Nano 33 IoT target microcontroller, real-time signal processing, an inter-controller communication link, wireless telemetry, and a host-side visualization system. The architecture supports both local execution of the filter on the ESP32-S3 and external execution on the target microcontroller, allowing the same platform to evaluate either implementation.

2.1 System Architecture

The platform consists of two embedded controllers: the ESP32-S3, which provides signal generation, control, telemetry, and communication functionality, and the Arduino Nano 33 IoT, used as the external target for filter evaluation. The overall architecture is shown in Figure 2.1.

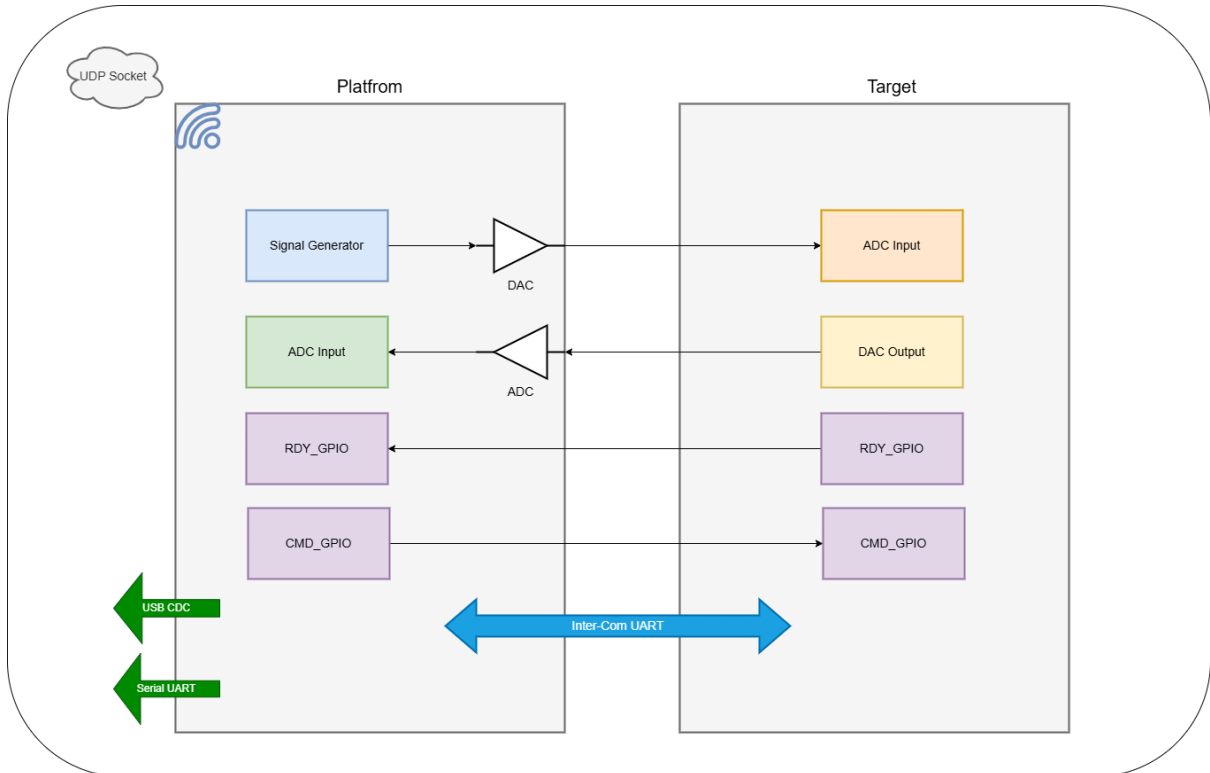


Figure 2.1: Overall architecture of the real-time digital filter evaluation platform.

The two controllers communicate over a UART-based interface with dedicated command and ready-handshake signals (Section 2.9), which allows the ESP32-S3 to control the target’s operating state and exchange configuration data at runtime. The ESP32-S3 encapsulates processed and acquired data in DSM packets (Section 2.10) and transmits them over UDP to the host application, which provides real-time visualization (Section 2.12).

2.2 ESP32-S3 Platform Controller

The ESP32-S3 is the central platform controller. Its firmware is implemented using ESP-IDF and FreeRTOS and performs:

- test-signal generation,
- acquisition of analog signals,
- execution of the digital filter in internal mode,
- communication with the external target,
- collection of target-side signals in external mode,
- runtime filter and signal configuration,
- DSM packet generation, and
- wireless UDP telemetry transmission.

The firmware is organized around a timer-driven processing loop: a hardware general-purpose timer generates the periodic sampling event, and the application task performs signal processing after receiving the timer notification. This keeps interrupt execution short and decouples the timing event from the processing workload.

2.3 Execution Modes

The platform supports two runtime-selectable execution modes.

Internal execution: the ESP32-S3 performs the complete signal-processing operation locally. On each timer event, the input is obtained from the configured signal generator or the ADC, and the selected digital filter processes the resulting sample (Section 2.6.1).

External execution: the ESP32-S3 does not perform the filter calculation. It instead coordinates the target microcontroller over the inter-controller interface, synchronizing on a GPIO handshake and sampling the target’s analog output directly rather than receiving a digital result over UART (Section 2.6.2).

The mode is switched at runtime: selecting external execution stops the internal ESP32-S3 processing timer, and reselecting internal execution restarts it. The same platform infrastructure therefore supports both local and target-side filter evaluation.

2.4 Test Signal Generation

The ESP32-S3 includes a configurable signal generator that provides deterministic test inputs, operating at the configured sampling frequency and supporting sinusoidal and impulse waveform modes. The default configuration uses a 5 kHz sampling frequency and a 100 Hz simulation frequency, with the default waveform set to impulse mode at amplitude 100 (both the ESP32-S3 and target firmware boot with this default; the frequency parameter applies to sine mode). Simulation input, frequency, amplitude, and waveform type are runtime-configurable without firmware modification.

Analog acquisition uses `ADC_UNIT_1` at 12 dB attenuation and the default ADC bit width. Channel 4 provides the primary input in both execution modes; channel 5 is used only in external mode, wired to the target’s DAC output for direct analog readback of the filtered result (Section 2.6.2).

2.5 Real-Time Sampling and Processing

The processing framework operates at a nominal sampling frequency $f_s = 5$ kHz, giving a nominal sampling period

$$T_s = \frac{1}{f_s} = \frac{1}{5000} = 200 \text{ } \mu\text{s}. \quad (2.1)$$

A hardware timer generates the periodic processing event. The timer alarm is programmed for $200 - 2 = 198 \text{ } \mu\text{s}$ rather than the nominal $200 \text{ } \mu\text{s}$, a fixed 2-tick reduction that compensates for ISR scheduling latency. The interrupt service routine performs only the minimum required operation — notifying the application task that a new sampling event is available — so that acquisition and filtering execute outside the ISR. For internal execution, the resulting sequence is:

$$\boxed{\text{Timer Event} \rightarrow \text{Input Acquisition} \rightarrow \text{Digital Filter} \rightarrow \text{Telemetry Generation}} \quad (2.2)$$

The measured execution time of the processing operation is recorded in the telemetry packet as the `dt` field, providing visibility of processing-time behaviour through the host interface.

2.6 Digital Filter Implementation

The platform supports eight filter configurations, selectable at runtime: floating-point and integer variants of direct-form IIR, FIR, second-order sections (SOS), and lattice-type filtering (LTC). The filter type is selected via the runtime command interface (Section 2.8), which passes the corresponding coefficient set to a common filter-initialization function shared by all eight configurations.

The coefficient bank implements a Bessel low-pass filter with a nominal cutoff frequency of 555 Hz at a 5 kHz sampling frequency, with both floating-point and fixed-point coefficient representations available. The default configuration is the floating-point IIR implementation.

Both the ESP32-S3 (internal mode) and the Arduino Nano 33 IoT (external mode) acquire their input via a 12-bit ADC and center the result to a signed representation using the same expression:

$$x[n] = ADC[n] - 2048. \quad (2.3)$$

The centred sample is passed to the selected filter:

$$y[n] = \text{Filter}(x[n]). \quad (2.4)$$

The two execution paths differ downstream of the filter, as described below.

2.6.1 Internal Processing (ESP32-S3)

In internal mode, the input sample (from the signal generator or ADC, Section 2.3) is centred using Eq. 2.3 and filtered using Eq. 2.4. The input and filtered samples are additionally converted to DAC-domain telemetry fields by reversing the centring offset:

$$\text{dac0} = x[n] + 2048, \quad \text{dac1} = y[n] + 2048. \quad (2.5)$$

Both signed and DAC-domain representations, together with the processing-time measurement, are stored in the telemetry structure. This path provides the local reference against which the external target implementation is evaluated.

2.6.2 External Target Processing (Arduino Nano 33 IoT)

In external mode, the ESP32-S3 does not receive the filtered sample digitally. It waits for the RDY line to fall, records the timestamp t_{low} , and samples two ADC channels: channel 4, on the shared input line ($x[n]$), and channel 5, wired to the target's analog DAC output, which yields the filtered result $y[n]$ directly as an analog readback. The ESP32-S3 then waits for RDY to return high, records t_{high} , and computes the target's per-sample processing latency:

$$dt = t_{\text{high}} - t_{\text{low}}. \quad (2.6)$$

Both channels are centred using Eq. 2.3 before insertion into the telemetry packet.

On the target itself, the Arduino Nano 33 IoT firmware implements a timer-driven acquisition and processing architecture, independent of the ESP32-S3's sampling instant: a hardware timer generates the sampling event, and the main application loop executes the processing chain

$$\boxed{\text{ADC} \rightarrow \text{Signal Centering} \rightarrow \text{Digital Filter} \rightarrow \text{DAC Scaling} \rightarrow \text{DAC}} \quad (2.7)$$

After filtering (Eq. 2.4), the result is converted from the signed 12-bit domain to the target's DAC output range, scaled while preserving the signal midpoint and clipped to the valid range:

$$\text{dac_out} = \text{clip}\left(\left\lfloor \frac{y[n] \cdot \text{DAC_MAX}}{4095} \right\rfloor + \frac{\text{DAC_MAX}}{2}, 0, \text{DAC_MAX}\right), \quad (2.8)$$

where `DAC_MAX` is the target's maximum DAC output code. This allows the target to execute the complete real-time processing chain independently of the host computer.

2.7 Target Operating States

The target microcontroller uses command and ready signals to coordinate two mutually exclusive states:

$$\boxed{\text{Sampling State}} \quad \boxed{\text{Configuration State}} \quad (2.9)$$

In the sampling state, the target executes the timer-driven acquisition, filtering, and DAC-update cycle. When a configuration command is received from the ESP32-S3, the target disables the sampling timer, enters the configuration state, and signals its state via the ready line. After the command is processed, the target re-enables the sampling timer and resumes real-time processing. This prevents configuration operations from executing concurrently with the real-time filtering sequence. The exact handshake protocol coordinating this transition is given in Section 2.9.

2.8 Runtime Filter Configuration

Filter reconfiguration without firmware recompilation is a core design requirement (Section 1.2). The ESP32-S3 exposes a runtime command interface for filter type, input source, simulation parameters, execution mode, and other processing parameters. The full command set is implemented identically on the ESP32-S3 and the target, except where noted:

Table 2.1: Runtime command interface of the ESP32-S3 platform.

Command	Effect
<code>fmode:<iiri iirf firi firf sosf fir digi tst off></code>	Select the filter type and numerical representation.
<code>imode:gpio / imode:<other></code>	Select the input source, such as the ADC input or the configured simulation input.
<code>freq:<hz></code>	Set the simulation signal frequency in Hz.
<code>amp:<n></code>	Set the simulation signal amplitude.
<code>wave:sin / wave:imp</code>	Select the simulation waveform as sinusoidal or impulse.
<code>scoeff:<type></code>	Stop the sampling timer and begin runtime coefficient upload for the selected filter type.
<code>gcoeff</code>	Print the currently active filter coefficients to the ESP32-S3 log.
<code>exemode:int / exemode:ext</code>	Request switching between internal ESP32-S3 execution and external target execution.
<code>stop / start</code>	Stop or restart the sampling timer.
<code>reset</code>	Restore the default filter, input source, simulation parameters, and execution configuration.
<code>ar</code>	Restart the ESP32-S3.

Table 2.2: Runtime command interface. `imode`, `freq`, `amp`, `wave`, and `gcoeff` apply to the ESP32-S3 only; the target implements `fmode`, `scoeff`, `reset`, and `stop/start`.

On receipt of a filter-selection command, the corresponding configuration is applied and the filter state is reinitialized. Coefficient upload (`scoeff:<type>`) additionally stops the sampling timer on both controllers and transfers the coefficient set as newline-terminated `w <key> <values...>` lines (e.g. `w iirf.b <21 floats>`, `w iirf.Nb <n>`), parsed identically on both firmwares. Once a complete, validated coefficient set is received, it is committed to the selected filter structure and the timer is restarted. This allows different coefficient configurations to be evaluated on the same hardware without a new firmware build.

2.9 Inter-Controller Communication

The ESP32-S3 drives CMD (GPIO4, active-low, default high) and reads RDY (GPIO5, pull-up, default high) from the target, over a secondary link (UART1, 115200 8N1). To issue a command, the ESP32-S3: waits for RDY low, drives CMD low, delays 5 ms, transmits the newline-terminated command line over UART1, then waits for RDY high before driving CMD back high:

$$\boxed{\text{RDY Low} \rightarrow \text{CMD Low} \rightarrow \text{Command Transfer} \rightarrow \text{RDY High} \rightarrow \text{CMD High}} \quad (2.10)$$

On the target, a falling CMD edge suspends the sampling timer and pulls RDY low. RDY returns high once a complete line has been received and the UART has been idle for 5 ms; the target resumes sampling once CMD returns high. This interface is not part of the high-rate signal-processing loop; it coordinates configuration and control between the two controllers.

2.10 DSM Telemetry Packet

After each valid processing cycle, the ESP32-S3 builds a DSM telemetry packet containing the input signal, generated signal, filter input, filter output, DAC-domain values, timing information (`dt`, Section 2.5), a sequence identifier, and an integrity field:

Field	Type	Notes
<code>sig</code>	<code>uint8</code>	Fixed signature, <code>0xAD</code>
<code>n</code>	<code>uint8</code>	Packet size (bits 0–5) and rolling 2-bit sequence counter (bits 6–7)
<code>seq</code>	<code>uint16</code>	UDP mode only
<code>adc_in, fgen_out, filt_x, filt_y, dac0, dac1, dt</code>	<code>int16 × 7</code>	Signal and timing fields
<code>crc16</code>	<code>uint16</code>	Integrity checksum

Table 2.3: DSM packet layout. The total packet size is 20 B in UDP mode (with `seq`) and 18 B in USB mode (without `seq`), as verified by a compile-time size assertion.

The sequence identifier allows packet tracking at the receiver. Packet integrity is provided by a 16-bit checksum computed as the bitwise XOR of all 16-bit words in the packet — despite the field name `crc16`, the algorithm is an XOR checksum rather than a cyclic redundancy check. The DSM packet is the common data representation between the embedded platform and the host-side visualization system.

2.11 UDP Telemetry Transmission

Rather than transmitting each DSM packet as an individual UDP datagram, packets are accumulated into batches of `UDP_BATCH_SIZE = 100`, prefixed by a 6-byte header (4-byte batch ID + 2-byte packet count), a size verified by a compile-time assertion:

$$\boxed{\text{Sampling} \rightarrow \text{DSM Packet} \rightarrow \text{Packet Batch} \rightarrow \text{UDP} \rightarrow \text{Host}} \quad (2.11)$$

Batching reduces per-datagram transmission overhead while retaining individual sample information within each DSM packet. The sequence number and batch identifier together let the host track the received telemetry stream and detect dropped batches (Section 3.2).

2.12 Host-Side Visualization

A Node.js-based host bridge receives the binary DSM telemetry over UDP and forwards the decoded data to a browser-based interface over WebSocket:

$$\boxed{\text{ESP32-S3} \rightarrow \text{UDP} \rightarrow \text{Node.js Bridge} \rightarrow \text{WebSocket} \rightarrow \text{Browser}} \quad (2.12)$$

The browser displays discrete samples using a staircase representation rather than an interpolated continuous line, preserving the discrete-time nature of the signals and making sample-to-sample variation, jitter, quantization effects, and limit-cycle behaviour more apparent. For each displayed signal, the interface computes the minimum, maximum, average, and peak-to-peak range, and separately displays the processing-time signal `dt_us` for direct visibility of timing behaviour.

Chapter 3

Results

3.1 Filter Design Verification

The designed ninth-order Bessel low-pass filter was first verified in MATLAB before implementation on the embedded platforms. The filter was designed for a nominal cutoff frequency of $f_c = 555$ Hz and a sampling frequency of $f_s = 5$ kHz.

The simulated magnitude response was evaluated to verify the required cutoff frequency and overall frequency-domain behaviour. The measured -3 dB cutoff frequency was

$$f_g = 554.75 \text{ Hz.} \quad (3.1)$$

The deviation from the specified cutoff frequency is therefore

$$\Delta f = |555 - 554.75| = 0.25 \text{ Hz,} \quad (3.2)$$

corresponding to a relative deviation of

$$\frac{0.25}{555} \times 100 = 0.045\%. \quad (3.3)$$

The measured cutoff frequency is therefore well within the specified $\pm 1\%$ tolerance.

The impulse-response analysis was additionally used to determine suitable truncation limits for the finite-word-length FIR implementations. The resulting coefficient decay was used to select the FIR lengths employed during the real-time evaluation.

3.2 Real-Time Visualization Interface

The implemented graphical interface provides real-time observation of the signals generated and processed by the embedded platforms. DSM telemetry is received by the host communication layer and displayed together with numerical signal statistics.

The waveform is rendered using a staircase representation rather than linear interpolation between consecutive samples. This preserves the discrete-time nature of the measured signals and makes sample-to-sample variations, quantization effects, timing jitter, and possible limit-cycle oscillations easier to identify.

The interface provides selectable channels for the input signal, generated signal, filter input, filter output, DAC signals, and the measured processing-time quantity `dt_us`.

For each displayed signal, the statistical table provides the minimum, maximum, arithmetic mean, and signal range:

$$\Delta x = x_{\max} - x_{\min}. \quad (3.4)$$

The processing-time channel `dt_us` is included in the same analysis to provide direct visibility of the execution-time behaviour during real-time operation.

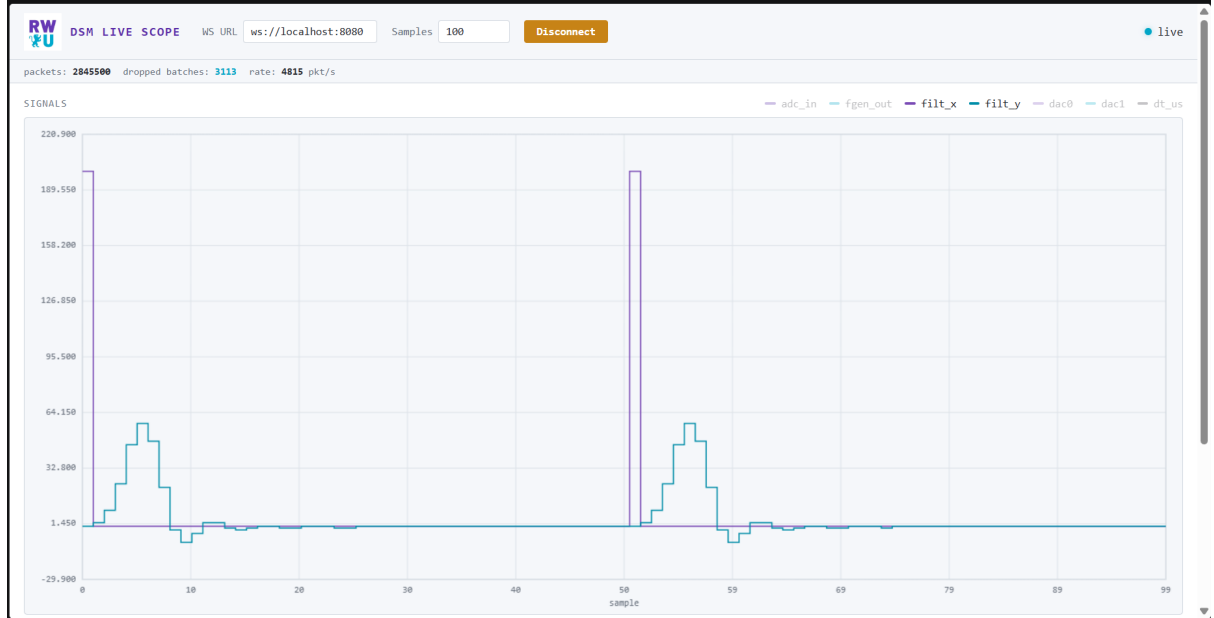


Figure 3.1: Real-time graphical interface for signal visualization and statistical analysis.

The visualization interface therefore provides both qualitative and quantitative observation of the filter response and its real-time execution behaviour.

3.3 Real-Time Filter Evaluation

The designed ninth-order Bessel low-pass filter was evaluated under real-time conditions on both the ESP32-S3 platform and the Arduino Nano 33 IoT target platform. The purpose of the hardware evaluation was to verify the functional behaviour of the different filter realizations and to investigate implementation-specific effects that may not be visible in the MATLAB simulation.

The evaluated filter structures comprise floating-point and integer variants of Direct Form I IIR, SOS, LTC, and FIR implementations. The same filter specification and test methodology were used for the different realizations.

The ESP32-S3 provides the reference real-time implementation for the platform, while the Arduino Nano 33 IoT provides an independent target environment for evaluating the filter implementation on a separate microcontroller.

A representative experiment using an impulse input is shown for the floating-point Direct Form I IIR implementation in Figure 3.2. The resulting response is consistent with the expected transient behaviour of the designed low-pass filter.

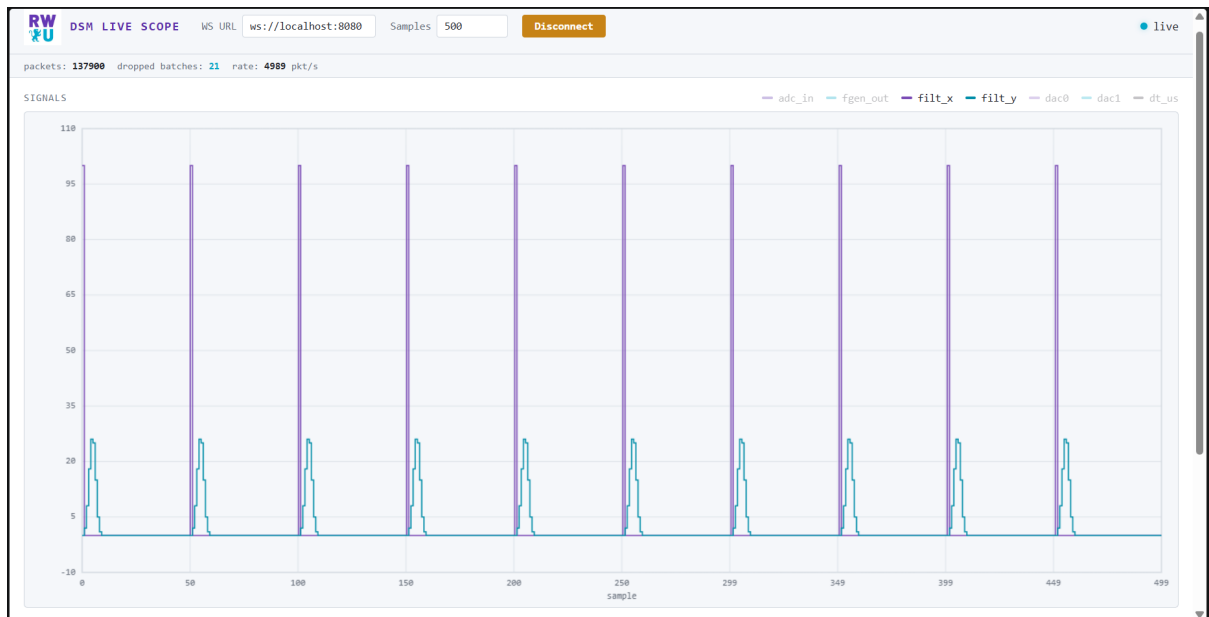


Figure 3.2: Real-time impulse response of the ninth-order floating-point Direct Form I IIR Bessel filter.

The input impulse reaches approximately 100, while the filtered output reaches a maximum of approximately 26 and subsequently decays over several samples. The staircase representation makes the discrete samples of the transient response clearly visible.

A zoomed view is shown in Figure 3.3.

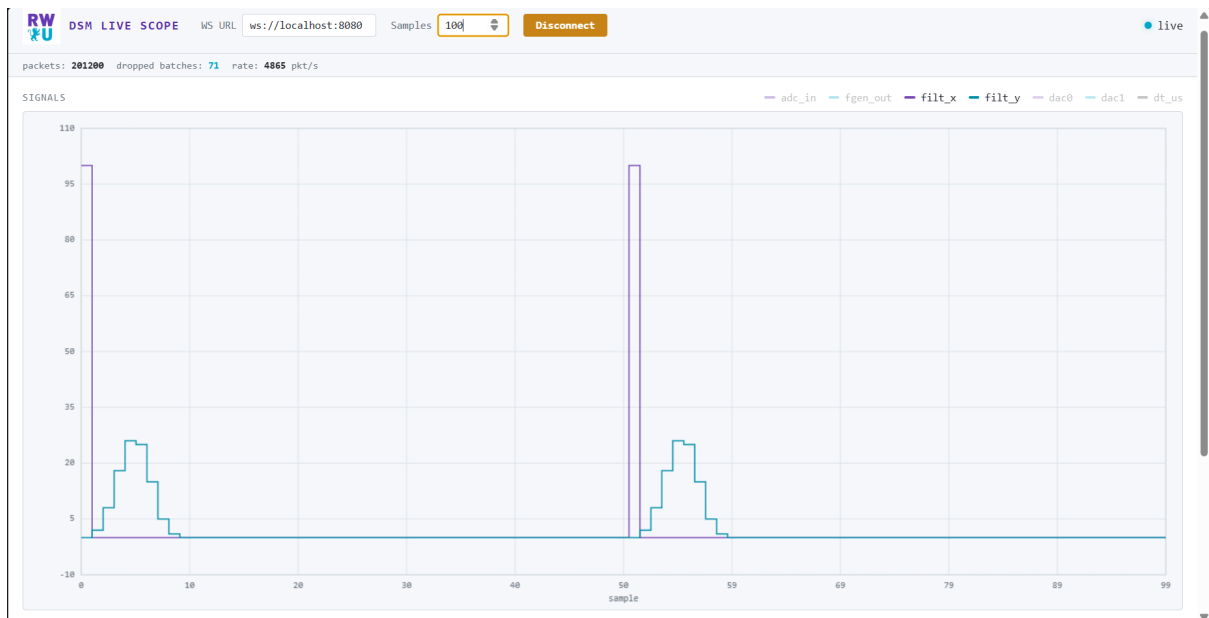


Figure 3.3: Zoomed view of the discrete-time impulse response of the floating-point Direct Form I IIR implementation.

The corresponding numerical signal statistics are shown in Figure 3.4. The measured filtered output has a maximum of approximately 26 compared with approximately 100 for the input impulse.

SIGNAL	MIN	MAX	AVERAGE	DELTA (MAX-MIN)
● <code>adc_in</code>	0	100	2	100
● <code>fgen_out</code>	0	100	2	100
● <code>filt_x</code>	0	100	2	100
● <code>filt_y</code>	0	26	2	26
● <code>dac0</code>	2048	2148	2050	100
● <code>dac1</code>	2048	2074	2050	26
● <code>dt_us</code>	6	107	7.990	101

Figure 3.4: Measured signal statistics during the floating-point IIR impulse-response test.

The same evaluation procedure was subsequently applied to the remaining filter realizations on the two embedded platforms. The results confirmed the expected filtering behaviour and demonstrated that the different structures could be executed under real-time conditions.

3.4 Limit-Cycle Behaviour

Particular attention was given to finite-word-length effects in the integer implementations. The Direct Form I IIR integer realization exhibited persistent limit-cycle oscillations for specific input amplitudes, with representative cases observed around amplitudes of 200 and 850.

This behaviour results from quantization effects within the recursive feedback structure. Although the corresponding floating-point filter is stable, finite-word-length arithmetic can produce a residual quantization error that is sustained by the feedback path.

In contrast, the tested SOS, LTC, and FIR integer implementations showed decay towards zero after the excitation was removed for input amplitudes up to 2000. Under the tested conditions, no persistent limit-cycle behaviour was observed for these structures.

The real-time experiments therefore demonstrated that the filter structure and numerical representation have a significant influence on the practical behaviour of a digital filter. This effect would not be fully apparent from the floating-point pole-zero analysis alone.

3.5 Execution Time and CPU Load

Execution-time behaviour was evaluated primarily on the ESP32-S3 platform. The ninth-order filter operates with a nominal sampling period of

$$T_s = \frac{1}{5000} = 200 \mu\text{s}. \quad (3.5)$$

The measured execution times and corresponding CPU-load estimates for the supported filter realizations are summarized in Table 3.1.

Table 3.1: Execution time and CPU load of the supported filter realizations on the ESP32-S3 for $N = 9$ and $T_s = 200 \mu\text{s}$.

Realization	Execution time (μs)	CPU load (%)
IIR float (Direct Form I)	6.5	3.25
IIR int (Direct Form I)	5.5	2.75
SOS float	4.5	2.25
SOS int	5.0	2.50
LTC float	5.0	2.50
LTC int	5.0	2.50
FIR float ($M = 23$)	5.0	2.50
FIR int ($M = 23$)	5.0	2.50

All evaluated realizations require substantially less than the available $200 \mu\text{s}$ sampling period on the ESP32-S3. The measured execution times therefore provide sufficient computational margin for real-time operation at the selected sampling frequency.

The processing-time values observed through the real-time visualization are consistent with this result. The `dt_us` channel provides direct visibility of the actual processing-time variation during operation.

3.6 Microcontroller Comparison

The real-time filter evaluation was performed on the ESP32-S3 and Arduino Nano 33 IoT. In addition to these two platforms, the Arduino Due and Teensy 4.1 were considered as reference microcontroller platforms for comparison of computational requirements.

The Arduino Nano 33 IoT was used as an external target platform for functional real-time filter evaluation. The present evaluation does not use the Arduino Due execution-time values as measurements of the Nano 33 IoT. Instead, the Due and Teensy values provide a reference for understanding how the computational requirements of the filter compare with other commonly used microcontroller platforms.

For the ninth-order filter, the reference results for the Arduino Due show that the absence of a hardware floating-point unit has a significant effect on floating-point execution. The floating-point Direct Form I IIR implementation requires approximately $131 \mu\text{s}$, corresponding to approximately 66% CPU load at a $200 \mu\text{s}$ sampling period. The integer SOS implementation requires approximately $71 \mu\text{s}$, corresponding to approximately 36% CPU load.

For comparison, the ESP32-S3 executes the same ninth-order realizations substantially faster. The floating-point Direct Form I IIR implementation requires approximately $6.5 \mu\text{s}$, corresponding to 3.25% CPU load, while the integer SOS implementation requires approximately $5 \mu\text{s}$, corresponding to 2.5% CPU load.

Table 3.2: Reference execution performance for selected ninth-order filter realizations on different microcontroller platforms.

Platform	IIR(float)	SOS(int)
Arduino Due	131 μ s (66%)	71 μ s (36%)
Teensy 4.1	25 μ s (13%)	25 μ s (13%)
ESP32-S3	6.5 μ s (3.25%)	5 μ s (2.5%)

The Arduino Due and Teensy 4.1 values in Table 3.2 are reference results from the accompanying microcontroller evaluation and should therefore be interpreted as comparative performance data rather than measurements of the Arduino Nano 33 IoT used as the external target in this project.

3.7 Comparison of Real-Time Realizations

The experimental results demonstrate two important aspects of real-time digital-filter implementation: computational efficiency and numerical robustness.

For floating-point arithmetic, the Direct Form I IIR realization provides a stable and computationally efficient implementation on the ESP32-S3. The measured execution time of approximately 6.5 μ s represents only 3.25% of the available 200 μ s sampling period.

For integer arithmetic, the Direct Form I IIR realization provides a slightly lower execution time but exhibits quantization-induced limit-cycle behaviour for specific input amplitudes. The SOS realization avoids this behaviour under the tested conditions while maintaining a low execution time of approximately 5 μ s on the ESP32-S3.

The results therefore demonstrate that computational speed alone is not sufficient for selecting a filter realization. Numerical robustness and the arithmetic representation must also be considered.

3.8 Discussion

The experimental results demonstrate that the designed ninth-order Bessel low-pass filter satisfies its frequency-domain specification and can be executed under real-time conditions on the developed embedded platforms.

The simulated cutoff frequency of 554.75 Hz differs from the target frequency of 555 Hz by only 0.045%, confirming the accuracy of the filter design. The pole analysis additionally confirms the stability of the floating-point filter.

The real-time experiments on the ESP32-S3 and Arduino Nano 33 IoT complement the MATLAB analysis by exposing implementation-specific effects. In particular, the Direct Form I integer realization exhibited limit-cycle oscillations for selected input amplitudes, despite the stable floating-point pole locations. The SOS, LTC, and FIR integer implementations showed improved behaviour under the tested conditions.

The real-time visualization was particularly useful for identifying these effects because the staircase representation preserves the discrete-time samples and makes residual oscillations directly visible. The statistical table and `dt_us` measurement provide additional quantitative information about the signal and processing behaviour.

The comparison with the Arduino Due and Teensy 4.1 further shows that the choice of microcontroller hardware has a significant influence on the computational feasibility of

a given filter realization. The ESP32-S3 provides substantial computational margin for the tested ninth-order filter, while the reference results for the Arduino Due show that floating-point processing can become a significant constraint when no hardware FPU is available.

3.9 Conclusion

The developed Real-Time Digital Filter Evaluation Platform successfully combines test-signal generation, embedded filter processing, telemetry transmission, and real-time visualization.

The ninth-order Bessel low-pass filter met its specified cutoff frequency with a deviation of only 0.045%. The different supported filter realizations were successfully evaluated under real-time conditions on the ESP32-S3 and Arduino Nano 33 IoT platforms.

The experiments demonstrated that floating-point Direct Form I IIR provides a stable and computationally efficient solution on the ESP32-S3. In contrast, the integer Direct Form I IIR implementation exhibited quantization-induced limit cycles for selected input conditions. The SOS, LTC, and FIR integer implementations showed improved numerical robustness under the tested conditions.

The comparison with the Arduino Due and Teensy 4.1 further demonstrates that the available arithmetic hardware has a direct impact on the suitability of a filter realization for real-time operation.

Consequently, the selection of a digital-filter implementation cannot be based solely on execution time. Filter structure, numerical representation, finite-word-length behaviour, and the computational capabilities of the target microcontroller must all be considered when selecting a realization for a real-time embedded application.